Implementation Document

for

Pennywise

Version 1.0

Prepared by

Group 11: **Group Name:** Infinite Loop

Anushika	**240315**	anushikad24@iitk.ac.in
Anushka Rajora	**240163**	anushkar24@iitk.ac.in
Gowra Akshitha	**240409**	gowrak24@iitk.ac.in
Chinthala Vinayasri	**240310**	chinthalav24@iitk.ac.in
Lingamsetti Rajasa	**240596**	lrajasa24@iitk.ac.in
Medini	**240645**	medinim24@iitk.ac.in
Raparthi Bhavishitha	**240852**	bhavisht24@iitk.ac.in
Anshika Singh	**240145**	anshikas24@iitk.ac.in
Nagella Venkata Snehitha	**240673**	snehithanv24@iitk.ac.in
Rachana Reddy V	**240822**	rachanav24@iitk.ac.in

Course: **CS253**

Mentor TA: ***Ujjwal Kajal***

Date: **27th March 2026**

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 3

APPENDIX A - GROUP LOG 4

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
v1.0	Chinthala Vinayasri, Daida Anushika, Raparthi Bhavishitha, Anushka Rajora, Gowra Akshitha, Lingamsetti Rajasa, Medini, Snehitha, Anshika, Rachana	The first version of the implementation document	27/03/26

● Implementation Details`

The software system is implemented using a combination of modern programming languages, frameworks, libraries, and database systems to ensure efficiency, scalability, and ease of development.

Programming Languages

The primary programming languages used in this project are:

JavaScript: Used for both frontend and backend development. It enables seamless communication between client and server.

SQL: Used for database queries and data management.

Justification:
JavaScript allows full-stack development using a single language, reducing complexity and improving productivity. SQL is essential for structured data storage and efficient querying.

Frameworks

Node.js: Used as the backend runtime environment.

Express.js: Used to build RESTful APIs for handling client requests.

Justification:
Node.js provides fast and scalable server-side execution, while Express.js simplifies API development with minimal setup and flexible routing.

Libraries

mysql: Used to connect Node.js with the MySQL database.

bcrypt: Used for password hashing to enhance security.

cors: Enables secure communication between frontend and backend.

dotenv: Manages environment variables securely.

Justification:
These libraries simplify development, improve security, and handle essential backend operations efficiently without building everything from scratch.

Database System

MySQL is used as a relational database system.

Justification:
MySQL is reliable, easy to use, and widely supported. It efficiently handles structured data such as users, categories, and transactions, and supports relationships using foreign keys.

Database Management Tool

phpMyAdmin is used to manage the database.

Justification:
phpMyAdmin provides a graphical interface, making it easy to create tables, run SQL queries, and manage data without needing advanced command-line knowledge.

Build Tools / Development Tools

XAMPP: Used to run the Apache server and MySQL locally.

VS Code: Used as the code editor.

Git & GitHub: Used for version control and team collaboration.

Justification:
XAMPP simplifies local server setup. VS Code provides a powerful and user-friendly coding environment. GitHub enables teamwork, version tracking, and easy project sharing.

System Architecture

The project follows a **client-server architecture**:

Frontend sends requests (API calls)

Backend (Node.js + Express) processes logic

Database (MySQL) stores and retrieves data

Justification:
This architecture ensures modularity, scalability, and clear separation of concerns.

Conclusion

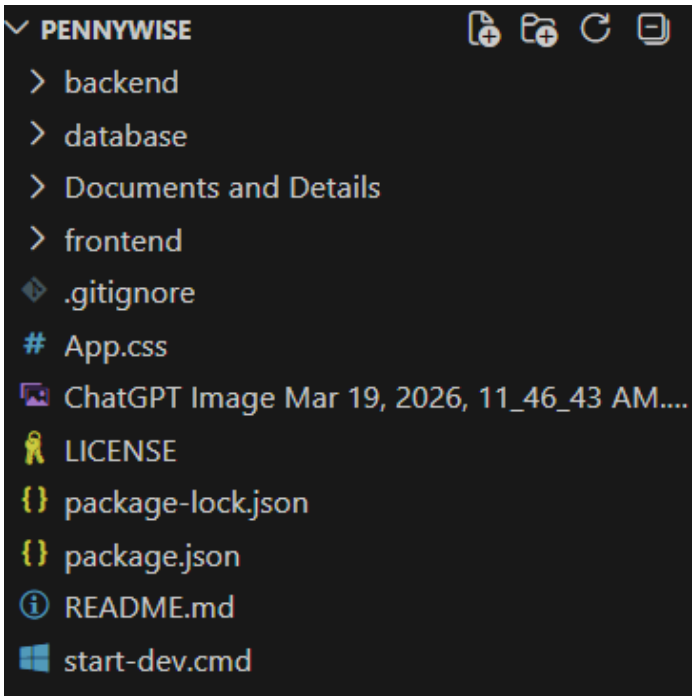
The selected technologies provide a balance of simplicity, performance, and scalability. They are widely used in industry and are suitable for building a secure and efficient expense tracking system.

• Codebase

2.1 Github Repository

GitHub Repository: https://github.com/anushkarajora11/Pennywise

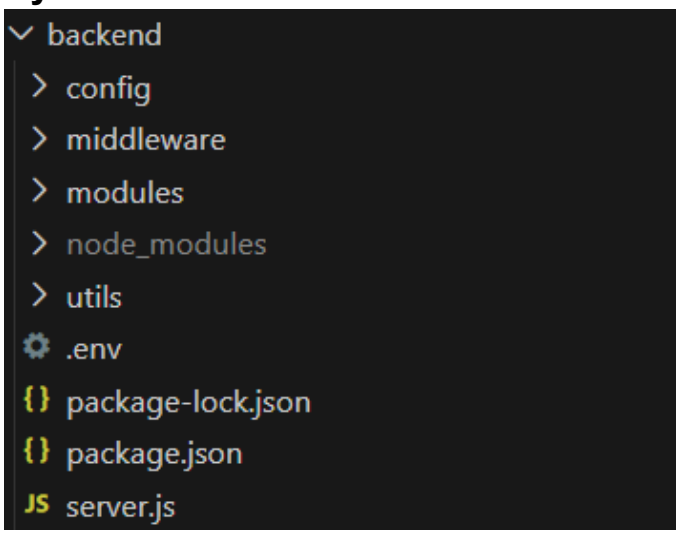
2.2 Codebase Structure



The PennyWise project is split into two main folders: frontend and backend.
This keeps the visual parts of the website separate from the database stuff.

The overall structure of the code, as displayed in the VSCode file explorer, consists of the following components:

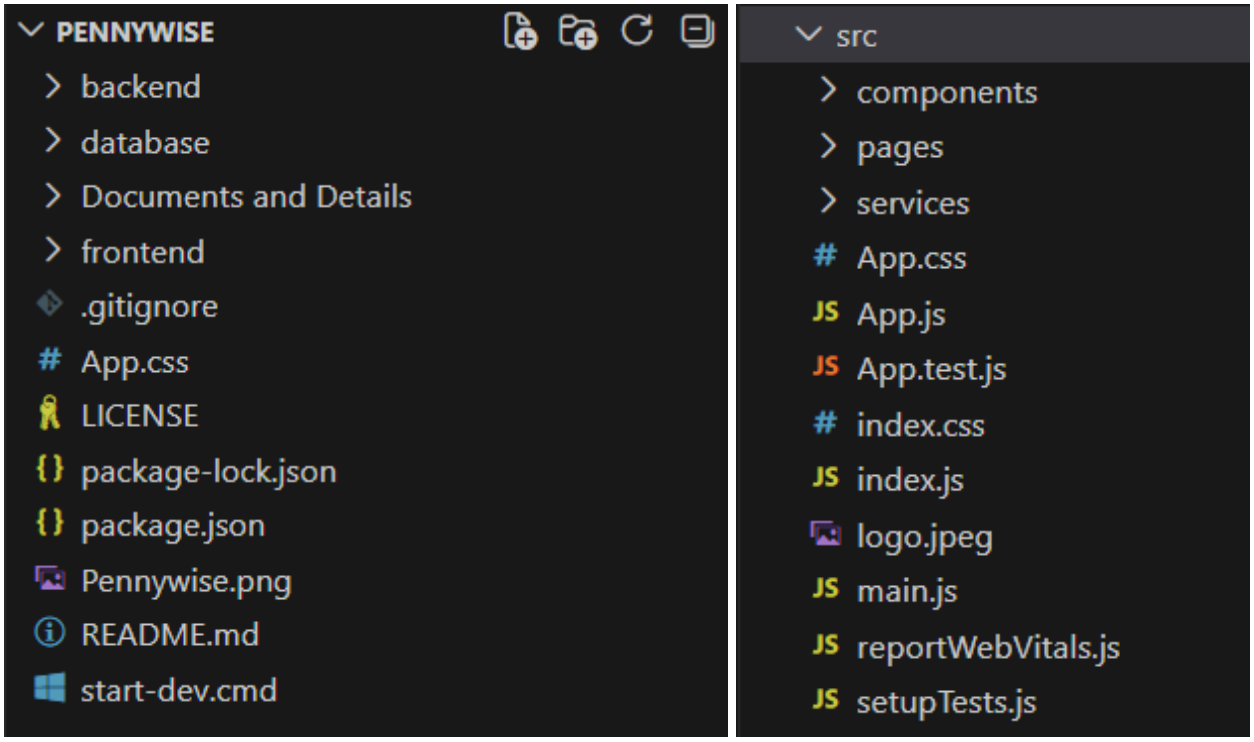
The Backend Subsystem



This folder runs the Node.js server. It handles the math and communicates with the MySQL database.

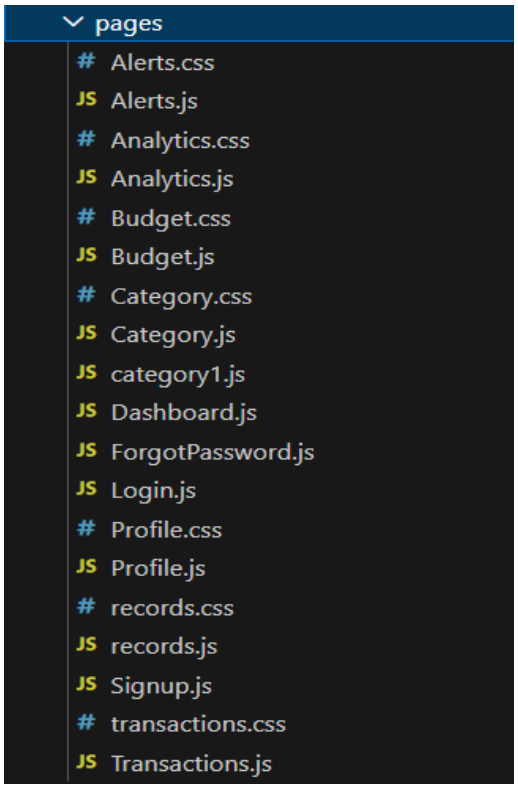
- **`server.js`:** This is the main file. It starts the server and sets up the basic links.
- **`config/db.js`:** This file connects the server to the MySQL database using the passwords in the `.env` file.
- **`controllers/`:** This is where the actual work happens.
  - `budgetController.js` has the code to add, edit, or delete budgets.
  - `analyticsController.js` does the math to add up expenses for the charts.
- **`routes/`:** This folder links the web addresses to the controllers. For example, it tells the server that going to `/budget/add` should trigger the code in `budgetController.js`.
- **`.env`:** A hidden file that stores private database passwords.
- **`package.json`:** A list of the extra tools the backend needs to run.

The Frontend Subsystem



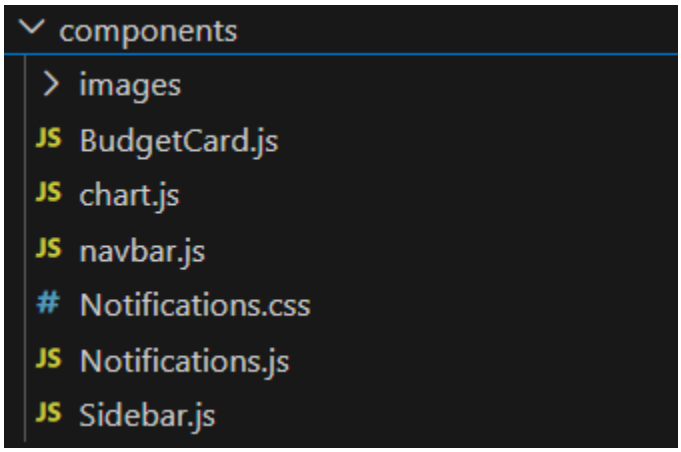
This folder runs the React app. It contains everything the user actually sees and clicks on.

- **`src/App.js`:** It looks at the web address and decides which page to show the user.
- **`src/pages/`:** This folder holds the main screens of the app.



- `Dashboard.js` is the home screen that shows a quick summary.
- `Analytics.js` draws the pie charts and line graphs.
- `Budget.js` is the screen where users can see and edit their budget limits.

- **`src/components/`:**



This folder holds smaller pieces of the screen that get used multiple times. For example, `Sidebar.js` is the menu on the side, and `Category.js` is the pop-up box for adding a new category.

- **`src/index.js`:** This file starts the React app and puts it on the screen.
- **`package.json`:** A list of the extra tools the frontend needs, like the chart drawing tool.

How it Works Together

Example: Adding a Budget

Here is what happens when a user adds a new budget:

- **User clicks save:** The user types a name and amount in the frontend form (`Category.js`) and clicks the save button.
- **Frontend sends data:** The React app sends that name and amount to the backend server.
- **Backend routes it:** The `server.js` file gets the data and passes it to `budgetRoutes.js`.
- **Backend saves it:** The router sends it to `budgetController.js`, which writes the new budget into the MySQL database.
- **Frontend updates:** The backend tells the React app that the save was successful. The React app then updates the screen to show the new budget.

2.3 Setting Up the Development Environment

Prerequisites

Before you start, make sure you have these three tools installed on your computer:

- **Node.js** (to run the Javascript code)
- **Git** (to download the code)
- **XAMPP** (to run the local MySQL database server)

Step 1: Set Up the Database

1. Open XAMPP and start both the **Apache** and **MySQL** modules.
2. Open your web browser and go to `http://localhost/phpmyadmin`.
3. Click **New** on the left menu and create a database named `expense_tracker`.
4. Click on `expense_tracker`, go to the **Import** tab at the top, upload the `expense_tracker.sql` file, and click **Go**.

Step 2: Start the Backend Server

1. Open your computer's terminal and download the code:

   ```
   git clone https://github.com/anushkarajora11/Pennywise.git
   ```

2. Move into the backend folder: `cd Pennywise/backend`
3. Install the required tools: `npm install`
4. Start the server by typing: `npm start` *(Leave this terminal window open. It runs on port 5001).*

Step 3: Start the Frontend Application

1. Open a new terminal window.

2. Move into the frontend folder: `cd Pennywise/frontend`
3. Install the required tools: `npm install`
4. Start the React website: `npm start`

Your browser will automatically open `http://localhost:3000` and the PennyWise dashboard will appear.

● Completeness

We were able to successfully implement nearly all the features specified in the Software Requirements Specification (SRS) for the Pennywise system. The system provides a comprehensive platform for personal finance management by integrating multiple modules such as authentication, transaction handling, budgeting, analytics, and alerts into a unified workflow. Each module has been implemented with a focus on usability, real-time updates, and seamless interaction with other components of the system. The implementation ensures that users can efficiently track, analyze, and control their financial activities.

3.1 Sign Up / Login

- We have implemented the Sign-Up and Login functionalities as described in the SRS, ensuring secure and reliable user authentication.
- During the sign-up process, users are required to provide essential details such as email, username, and password. This ensures that each user account is uniquely identifiable within the system.
- An **OTP-based verification mechanism** is integrated into the registration process. The system generates a one-time password and sends it to the user's email, which must be entered correctly to complete the registration. This prevents unauthorized or fake account creation.
- The login functionality allows users to securely access their accounts using their registered credentials. Proper validation checks are performed to ensure correctness of input data.
- The system uses **secure password storage techniques**, such as hashing, to protect user credentials and prevent exposure of sensitive information.
- Session management is implemented to maintain user authentication across different pages. Once logged in, users can navigate the application without needing to re-enter credentials.
- Error handling mechanisms are in place to handle invalid login attempts, ensuring a smooth and secure user experience.

3.2 Profile

- This page is accessible only after login.
- Users can view personal details such as username and email.
- The system allows editing and updating of profile information.
- Changes made by the user are stored and reflected immediately in the system.
- Input validation is applied to ensure correctness and prevent invalid data entry.
- A **Reset option is implemented**, allowing users to reset their account credentials (such as password) securely in case they forget or need to update them.
- The reset functionality is designed with proper verification mechanisms to ensure that only authorized users can perform sensitive updates.
- This module ensures that users maintain accurate, secure, and up-to-date account information.

3.3 Home Page / Records

- Upon successful login, users are redirected to the **Records (Dashboard)** page, which acts as the central interface of the application.
- The dashboard displays key financial indicators such as:
  - Total Income
  - Total Expenses
  - Current Balance
- These values are dynamically computed based on the transactions stored in the system and are updated in real time whenever new transactions are added, edited, or deleted.
- A **monthly filter** is implemented, allowing users to view financial data for a specific time period. This helps in analyzing monthly spending patterns.
- The records section presents a structured and organized list of transactions, including:
  - Date of transaction
  - Category or description
  - Amount
  - Day
  - Time
- The system ensures clarity by distinguishing income and expense entries visually.
- In cases where no transactions exist, a clear message such as **"No transactions yet"** is displayed, improving usability and guiding the user.
- A prominently placed **"+ Add Transaction" button** enables quick access to the transaction entry interface.
- The dashboard serves as a **data integration hub**, connecting transaction data with analytics and budget modules.

3.4 Transaction Management

- The system includes a robust **Transaction Management module**, allowing users to record and manage financial activities efficiently.
- Users can add both **income and expense transactions**, making the system flexible for various financial scenarios.
- The Add Transaction interface allows users to input:
  - Transaction type (Income / Expense)
  - Category (Food, Travel, Shopping, etc.)
  - Amount
  - Optional notes or descriptions
- Each transaction is stored in the database with detailed attributes such as date, category, amount, and type.
- The system maintains a **chronological transaction history**, enabling users to track their financial activities over time.
- Users can edit transactions to correct mistakes or update details.
- Users can also delete transactions, ensuring control over stored data.
- Every transaction automatically updates:
  - Dashboard summaries
  - Budget tracking values
  - Analytics graphs
- This ensures consistency and eliminates redundancy in data processing.

3.5 Category Management

- The Category Management module provides a structured way to organize financial transactions by classifying them into meaningful groups such as Food, Travel, Shopping, Utilities, and Entertainment.
- The system includes a set of predefined categories to cover common use cases, ensuring that users can quickly start recording transactions without additional setup.
- In addition to default categories, users are given the flexibility to create custom categories based on their personal spending habits, making the system adaptable to different financial behaviors.
- Each transaction is mandatorily linked to a category, ensuring that all financial data is properly classified and can be used effectively in analytics and budgeting.
- The system ensures tight integration between categories and transactions, so that any categorization is consistently reflected across all modules including charts and budgets.
- Users are allowed to modify existing categories when required, enabling long-term usability as financial needs evolve.
- This module plays a foundational role in the system by improving data organization, which directly enhances the accuracy and usefulness of analytics and budget tracking.

3.6 Budget Management

- The Budget Management module enables users to define financial limits for different categories, helping them plan and control their expenses effectively.
- Users can create budgets using the “Add Budget” feature, where each budget is associated with a specific category such as Food, Travel, or Shopping.
- The system provides clear guidance when no budgets are present by displaying user-friendly messages, encouraging users to begin budget tracking.
- The module supports both predefined and custom categories, ensuring flexibility in financial planning.
- Once a budget is created, the system continuously tracks spending against it by integrating real-time transaction data.
- It calculates and displays key parameters such as the total budget amount, amount spent, remaining balance, and percentage of budget utilization.
- These values are updated dynamically whenever transactions are added, edited, or deleted, ensuring that users always see the latest financial state.
- Visual indicators such as progress bars are used to represent budget consumption, making it easier for users to understand their spending at a glance.
- Users are allowed to edit budget values at any time, allowing them to adjust their financial plans according to changing needs.
- The integration between budgets and transactions ensures that all calculations remain consistent and accurate.
- This module helps users maintain financial discipline by providing continuous feedback on their spending behavior.

3.7 Charts / Analysis

- The Charts and Analysis module provides a comprehensive visualization system that converts raw financial data into meaningful insights.
- It enables users to analyze their financial activities through graphical and summary-based representations.
- The system supports both monthly and yearly views, allowing users to examine short-term and long-term financial trends.
- A flexible filtering mechanism is implemented, enabling users to select specific months and years for detailed analysis.

Graphical Representation

- A line graph is implemented to display expense trends over time, helping users identify patterns such as increases or decreases in spending.
- The graph reflects daily expense variations within a selected time period, providing a clear temporal understanding of financial behavior.
- Users can switch between different visualization modes, including line graphs for trend analysis and pie charts for category distribution.
- This dual representation ensures that users can analyze both time-based trends and proportional spending.

Category-wise Analysis

- The system provides a category-wise breakdown of expenses, allowing users to see how much they spend in each category.
- Visual elements such as horizontal bars or proportional indicators are used to represent category-wise spending.
- This helps users quickly identify high-spending areas and take corrective actions.

Summary Representation

- A summary panel displays key financial metrics such as total income, total expenses, and savings.
- These values provide a quick snapshot of the user's financial health.
- The summary is dynamically updated based on transaction data.

Export Functionality

- The system includes an Export CSV feature that allows users to download their transaction data.
- This enables offline analysis, record keeping, and integration with other financial tools.

- The entire analytics module is interactive, allowing users to dynamically change filters, switch views, and explore data without reloading the application.
- This module plays a crucial role in helping users make informed financial decisions by providing clear and actionable insights.

3.8 Alerts

- The Alerts module provides a real-time monitoring system that tracks user spending and budget usage continuously.
- It is tightly integrated with both the transaction and budget modules, ensuring that alerts are based on accurate and updated data.

Budget Monitoring

- The system displays a detailed overview of budget usage, including total allocated budget, current spending, and remaining balance.
- It also calculates and shows the percentage of budget utilization.
- A visual progress bar is used to represent how much of the budget has been consumed.
- Users can edit budget values directly from the alerts interface, improving usability.

Threshold Alerts

- The system allows users to define custom alert thresholds based on percentage of budget usage (e.g., 80%, 90%).
- Multiple thresholds can be configured to provide layered warnings as spending increases.
- This feature helps users receive early warnings before exceeding their budget.

Alert Generation

- Alerts are generated when spending reaches predefined threshold levels.
- Alerts are also triggered when the budget is exceeded or when overall savings fall below acceptable levels.
- These alerts are updated dynamically as transactions are added or modified.

Integration

- The alerts system is directly connected to transaction data and budget values.
- Any change in transactions or budgets is immediately reflected in alert calculations.
- This ensures consistency and real-time responsiveness across the system.
- The Alerts module helps users identify excessive spending patterns and take corrective actions.

3.9 Overall System

- The system successfully integrates all major modules into a unified financial management platform.
- Each module, including transactions, categories, budgets, analytics, and alerts, is interconnected and shares data seamlessly.
- The system supports a complete workflow:
    - Add transactions
    - Categorize them
    - Track financial activity

    - Analyze trends
    - Set budgets
    - Receive alerts
- All modules operate in real time, ensuring that updates in one module are immediately reflected in others.
- For example, adding a transaction updates dashboard summaries, modifies budget usage, updates analytics graphs, and may trigger alerts.
- The system provides an interactive and user-friendly interface with features such as filters, graph toggling, CSV export, and dynamic updates.
- The implementation ensures accuracy, consistency, and efficiency in managing financial data.
- Overall, the system fulfills the objectives defined in the SRS and provides a complete and practical solution for personal finance management.
    -
- This results in a comprehensive and user-friendly financial management solution.

3.10 Future Implementations

- Integration of intelligent financial analysis features such as automated detection of spending patterns (e.g., high food expenses) and personalized suggestions for budgeting and saving.
- Integration with banking systems to automatically import transaction data and reduce manual entry.
- Development of a mobile application to improve accessibility and provide real-time notifications.
- Enhancement of security through features such as two-factor authentication and improved encryption mechanisms.
- Expansion of the alert system to include customizable thresholds and external notifications via email or SMS.
- Addition of report export functionality for generating financial summaries in formats such as PDF and Excel.
- Improvements in user interface design, including better responsiveness, theme customization, and enhanced usability.
- Scalability improvements to support larger datasets and multiple users efficiently.
- Introduction of collaborative features such as shared budgets and group financial tracking.

Appendix A - Group Log

SL. No.	Date	Timings	Venue	Description
1	12/03/2026	9:00 pm - 11:00 pm	Hall 6	Discussion on website plan and divided work among team members
2	13/03/2026	7:00pm-8:30pm	KD LAB	Completed database design and uploaded the structure in the Github
3	17/ 03/2026	2:00pm-5:00pm	RM	Reviewed and discussed individual progress on assigned project modules.
4	23/03/2026	7:30pm-9:00pm	KD LAB	Initiated work on the implementation document after detailed discussion.
5	25/03/26	6:00pm-7:00pm	Hall 6	Reviewed and discussed individual progress on assigned project work
6	26/03/26	9:00pm-11:00pm	RM	Discussed and Completed the implementation document after thorough review and refinement.